

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №4 по курсу
«Операционные системы»

Группа: М8О-213Б-23

Студент: Османова В.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 28.12.24

Москва, 2024

Постановка задачи

Вариант 9.

Требуется создать две динамические библиотеки, реализующие два аллокатора, соответственно. Библиотеки загружаются в память с помощью интерфейса ОС (dlopen / LoadLibrary) для работы с динамическими библиотеками. Выбор библиотеки, реализующей аллокатор, осуществляется чтением первого аргумента при запуске программы (argv[1]). Этот аргумент должен содержать путь до динамической библиотеки (относительный или абсолютный). Если аргумент не передан или по переданному пути библиотеки не оказалось, то указатели на функции, реализующие API аллокатора ниже, должны быть присвоены функциям, которые оборачивают системный аллокатор ОС (mmap / VirtualAlloc) в этот API. Эти аварийные оберточные функции должны быть реализованы внутри программы, которая загружает динамические библиотеки. Каждый аллокатор памяти должен иметь функции аналогичные стандартным функциям malloc и free (realloc, опционально). Перед работой каждый аллокатор инициализируется свободными страницами памяти, выделенными стандартными средствами ядра (mmap / VirtualAlloc). Необходимо самостоятельно разработать стратегию тестирования для определения ключевых характеристик аллокаторов памяти. При тестировании нужно свести к минимуму потери точности из-за накладных расходов при измерении ключевых характеристик, описанных выше.

Алгоритмы аллокаторов: списки свободных блоков (наиболее подходящее) и алгоритм двойников.

Общий метод и алгоритм решения

Использованные системные вызовы:

- void *mmap(void *addr, size_t len, int prot, int flags, int fildes, off_t offset) – выделение памяти
- int munmap(void *addr, size_t len) – освобождение памяти

Аллокатор на линейном списке:

Память делится на блоки произвольного размера, каждый блок начинается и заканчивается отметкой типа size_t, которая содержит размер этого блока, младший бит отметки является флагом занятости блока. Имея ссылку на один любой блок, мы можем получить доступ только к следующему и предыдущему блокам, поэтому эта структура является линейным списком. При инициализации аллокатора память размечается на один блок и блок-терминатор в конце доступной памяти. При аллокации мы выбираем блок, размер которого достаточен для выделения, при этом минимален среди других подходящих блоков. Если найденный блок больше необходимого, и разница достаточна, чтобы разместить на ней метки начала и конца блока, то блок дробится на 2, и занимает только первый, иначе занятым считается весь блок. При освобождении памяти проверяются соседние блоки, если среди них есть свободные, блоки сливаются.

Аллокатор алгоритмом двойников:

Блоки размера 2^n , метка начала и конца блока содержит число n и флаг занятости. Т.к. размер памяти ограничен типом size_t, степень двойки не превышает 32 и для её хранения достаточно метки типа char. Отметки включены в те 2^n байт, для того чтобы блоки можно было делить на меньшие, поэтому каждый блок содержит $2^n - 2$ байта памяти. При

инициализации аллокатора память размечается на 1 блок максимального возможного размера и терминатор. При аллокации определяется нужный размер блока, далее происходит поиск минимального блока со степенью равной или большей нужной. При необходимости блок дробится на меньшие. При освобождении памяти определяется на смещение от начала выделенной памяти в байтах, двоичное представление этого числа помогает определить с какой стороны необходимо искать двойника, если двойник свободен, то происходит слияние, процесс продолжается до тех пор, пока двойник не окажется занят или пока мы не освободим всю память.

Код программы

main.c

```
#include <unistd.h>
#include <dlfcn.h>
#include <sys/mman.h>
#include <string.h>
#include "allocator.h"

static allocator_create_func *allocator_create;
static allocator_destroy_func *allocator_destroy;
static allocator_alloc_func *allocator_alloc;
static allocator_free_func *allocator_free;

void print(char* msg) {
    write(STDOUT_FILENO, msg, strlen(msg));
}

Allocator* create_impl(void *const memory, const size_t size) {
    (void) memory;
    (void) size;
    return NULL;
}

void destroy_impl(Allocator *const allocator) {
    (void) allocator;
}

void* alloc_impl(Allocator *const allocator, const size_t size) {
    (void) allocator;
    void* mem = mmap(0, size + sizeof(size_t), PROT_READ | PROT_WRITE, MAP_ANONYMOUS | MAP_SHARED, -1, 0);
    if (mem == MAP_FAILED) {
        return NULL;
    }
    size_t* size_ptr = (size_t*) mem;
    *size_ptr = size;
    return mem + sizeof(size_t);
}

void free_impl(Allocator *const allocator, void *const memory) {
    (void) allocator;
    if (memory == NULL) {
        return;
    }
    void* mem = memory - sizeof(size_t);
    munmap(mem, *((size_t*) mem));
}

int main (int argc, char* argv[]) {
    void *library = dlopen(argv[1], RTLD_LOCAL | RTLD_NOW);
```

```

if ((argc > 1) && library) {
    allocator_create = dlsym(library, "allocator_create");
    if (allocator_create == NULL) {
        char* msg = "fail to find create function implementation\n";
        write(STDOUT_FILENO, msg, strlen(msg));
        allocator_create = &create_impl;
    }
    allocator_destroy = dlsym(library, "allocator_destroy");
    if (allocator_create == NULL) {
        char* msg = "fail to find destroy function implementation\n";
        write(STDOUT_FILENO, msg, strlen(msg));
        allocator_destroy = &destroy_impl;
    }
    allocator_alloc = dlsym(library, "allocator_alloc");
    if (allocator_create == NULL) {
        char* msg = "fail to find alloc function implementation\n";
        write(STDOUT_FILENO, msg, strlen(msg));
        allocator_alloc = &alloc_impl;
    }
    allocator_free = dlsym(library, "allocator_free");
    if (allocator_create == NULL) {
        char* msg = "fail to find free function implementation\n";
        write(STDOUT_FILENO, msg, strlen(msg));
        allocator_free = &free_impl;
    }
} else {
    char* msg = "fail to open library, standard functions is being used\n";
    write(STDOUT_FILENO, msg, strlen(msg));
    allocator_create = &create_impl;
    allocator_destroy = &destroy_impl;
    allocator_alloc = &alloc_impl;
    allocator_free = &free_impl;
}
print("start\n");
void* memory = mmap(0, 1 << 16, PROT_READ | PROT_WRITE, MAP_ANONYMOUS | MAP_SHARED, -1, 0);
print("mapped\n");
Allocator* all = allocator_create(memory, 1 << 16);
print("allocator created\n");
char* arr1 = allocator_alloc(all, 20);
print("arr1 allocated\n");
char* arr2 = allocator_alloc(all, 10);
print("arr2 allocated\n");
allocator_free(all, arr1);
print("arr1 freed\n");
arr1 = allocator_alloc(all, 12);
print("arr1 allocated\n");
allocator_free(all, arr1);
allocator_free(all, arr2);
allocator_destroy(all);
munmap(memory, 1 << 16);
if (library) {
    dlclose(library);
}
print("all is well\n");
return 0;
}

```

allocator1.c

```

#include "allocator.h"
#include <sys/mman.h>

```

```

//list

```

```

Allocator* allocator_create(void* const memory, const size_t size) {
    Allocator *allocator = (Allocator*) memory;
    size_t *start = (size_t*)(memory + sizeof(Allocator));
    size_t block_size = size - 3 * sizeof(size_t) - sizeof(Allocator);
    *start = block_size << 1; // в начале блока записан его размер
    *(size_t*)((void*)start + block_size) = *start; // в конце блока он продублирован
    *(size_t*)((void*)start + block_size + sizeof(size_t)) = 0; // после всех блоков находится терминатор
    allocator->ptr = memory + sizeof(Allocator);
    return allocator;
}

void allocator_destroy(Allocator *const allocator) {
    allocator->ptr = NULL;
}

void* allocator_alloc(Allocator *const allocator, const size_t size) {
    size_t* most_suitable = NULL;
    size_t most_suitable_size = 0;
    size_t* current = allocator->ptr;
    size_t cur_size = *current >> 1;
    while(*current) {
        if ((*current) & 1) {
            current = (size_t*)((void*)current + cur_size + 2 * sizeof(size_t));
            cur_size = *current >> 1;
            continue;
        }
        cur_size = *current >> 1;
        if ((cur_size > size) && (!most_suitable) || (most_suitable_size > cur_size)) {
            most_suitable = current;
            most_suitable_size = cur_size;
        }
        current = (size_t*)((void*)current + cur_size + 2 * sizeof(size_t));
    }
    if (!most_suitable) {
        return NULL;
    }
    if ((long)(most_suitable_size - size - 2 * sizeof(size_t)) > 0) {
        *most_suitable = (size << 1) | 1; //new block start
        *(size_t*)((void*)most_suitable + size + sizeof(size_t)) = (size << 1) | 1; //new block end
        *(size_t*)((void*)most_suitable + size + 2 * sizeof(size_t)) = (most_suitable_size - size - 2 * sizeof(size_t)) << 1;
        //empty block start
        *(size_t*)((void*)most_suitable + most_suitable_size + sizeof(size_t)) = (most_suitable_size - size - 2 * sizeof(size_t))
        << 1; //empty block end
    } else {
        *most_suitable = *most_suitable | 1; // cannot split block
        *(size_t*)((void*)most_suitable + most_suitable_size + sizeof(size_t)) = *most_suitable | 1;
    }
    void* ptr = (void*)most_suitable + sizeof(size_t);
    return ptr;
}

void allocator_free(Allocator *const allocator, void *const memory) {
    size_t* to_free = (size_t*)(memory - sizeof(size_t));
    size_t to_free_size = (*to_free) >> 1;
    *to_free = *to_free & ~1;
    size_t* to_free_end = (size_t*)(memory + to_free_size);
    *to_free_end = *to_free;
    if ((to_free != allocator->ptr) && !(*to_free - 1 & 1)) { //left is empty
        size_t left_size = *(to_free - 1) >> 1;
        to_free = (size_t*)((void*)to_free - 2 * sizeof(size_t) - left_size);
        *to_free = (to_free_size + left_size + 2 * sizeof(size_t)) << 1;
        *to_free_end = (to_free_size + left_size + 2 * sizeof(size_t)) << 1;
    }
}

```

```

}
if ((*to_free_end + 1) == 0) && !(*to_free_end + 1) & 1) { //right is empty
    size_t right_size = *(to_free_end + 1) >> 1;
    to_free_end = (size_t*)((void*)to_free_end + 2 * sizeof(size_t) + right_size);
    *to_free = (to_free_size + right_size + 2 * sizeof(size_t)) << 1;
    *to_free_end = (to_free_size + right_size + 2 * sizeof(size_t)) << 1;
}
}
}

```

Allocator2.c

```

#include "allocator.h"
#include <sys/mman.h>

```

```

//buddy

```

```

Allocator* allocator_create(void* const memory, const size_t size) {
    Allocator *allocator = (Allocator*) memory;
    char *start = (char*)(memory + sizeof(Allocator));
    size_t block_size = size - 1 - sizeof(Allocator);
    char block_rang = 0;
    while (block_size > 1) {
        block_rang++;
        block_size >>= 1;
    }
    *start = block_rang << 1; // в начале блока записан его размер
    *(size_t*)((void*)start + (1 << block_rang) - 1) = *start; // в конце блока он продублирован
    *(size_t*)((void*)start + (1 << block_rang)) = -1; // после всех блоков находится терминатор
    allocator->ptr = memory + sizeof(Allocator);
    return allocator;
}

```

```

void allocator_destroy(Allocator *const allocator) {
    allocator->ptr = NULL;
}

```

```

void* allocator_alloc(Allocator *const allocator, const size_t size) {
    char* most_suitable = NULL;
    char most_suitable_rang = 0;
    char* current = allocator->ptr;
    char cur_rang = *current >> 1;
    char need_size = size + 2;
    char need_rang = 0;
    while (need_size > 1) {
        need_rang++;
        need_size >>= 1;
    }
    while(*current != -1) {
        if ((*current) & 1) {
            current = (current + (1 << cur_rang));
            cur_rang = *current >> 1;
            continue;
        }
        cur_rang = *current >> 1;
        if ((cur_rang >= need_rang) && ((!most_suitable) || (most_suitable_rang > cur_rang))) {
            most_suitable = current;
            most_suitable_rang = cur_rang;
        }
        current = (current + (1 << cur_rang));
    }
    if (!most_suitable) {
        return NULL;
    }
}

```

```

cur_rang = most_suitable_rang;

while (cur_rang != need_rang) {
    cur_rang--;
    *most_suitable = cur_rang << 1;
    *(most_suitable + (1 << cur_rang) - 1) = cur_rang << 1;
    *(most_suitable + (1 << cur_rang)) = cur_rang << 1;
    *(most_suitable + (1 << (cur_rang + 1)) - 1) = cur_rang << 1;
}
*most_suitable = *most_suitable | 1;
*(most_suitable + (1 << need_rang) - 1) = *most_suitable | 1;

void* ptr = (void*)most_suitable + 1;
return ptr;
}

void allocator_free(Allocator *const allocator, void *const memory) {
    char* to_free = memory - 1;
    char to_free_rang = (*to_free) >> 1;
    *to_free = *to_free & ~1;
    char* to_free_end = (memory + (1 << to_free_rang) - 1);
    *to_free_end = *to_free;
    size_t location = 0;
    char* temp = allocator->ptr;
    while (temp != to_free) {
        temp = temp + 1;
        location++;
    }
    location >>= 2;
    char cur_rang = to_free_rang;
    while(1) {
        if (!*(to_free + (1 << cur_rang)) && !(location & 1)) { //terminator and block is left
            break;
        }
        if (location & 1) { //block is right
            if (!*(to_free - 1) & 1) { //left block is not empty
                break;
            }
            *(to_free - (1 << cur_rang)) = (cur_rang + 1) << 1;
            *(to_free + (1 << cur_rang) - 1) = (cur_rang + 1) << 1;
            cur_rang++;
        } else { //block is left
            if (!*(to_free + (1 << cur_rang)) & 1) { //right block is not empty
                break;
            }
            *to_free = (cur_rang + 1) << 1;
            *(to_free + (1 << (cur_rang + 1)) - 1) = (cur_rang + 1) << 1;
            cur_rang++;
        }
        location >>= 1;
    }
}

```

Протокол работы программы

Тестирование:

```

$ ./main ./allocator2.so
start

```



```

)          = 7
write(1, "allocator created\n", 18allocator created
)          = 18
write(1, "arr1 allocated\n", 15arr1 allocated
)          = 15
write(1, "arr2 allocated\n", 15arr2 allocated
)          = 15
write(1, "arr1 freed\n", 11arr1 freed
)          = 11
write(1, "arr1 allocated\n", 15arr1 allocated
)          = 15
munmap(0x7ff873191000, 65536)      = 0
munmap(0x7ff8731af000, 16400)     = 0
write(1, "all is well\n", 12all is well
)          = 12
exit_group(0)                    = ?
+++ exited with 0 +++

```

Вывод

В ходе выполнения задачи была успешно разработана система с двумя динамическими библиотеками, каждая из которых реализует собственный аллокатор памяти. Программа обеспечивает удобный интерфейс для загрузки этих библиотек на основе переданного при запуске аргумента, а также корректно обрабатывает ситуации, когда библиотека недоступна, предоставляя альтернативные решения через стандартный системный аллокатор.